# An Open-source Repository Retrieval Service Using Functional Semantics for Software Developers

Jiawei Wu, Yanchun Sun*, Jiaqi Zhang
*Key Laboratory of High Confidence Software Technologies, Ministry of Education*
*School of Computer*
*Peking University*
Beijing, China
{jw_wu, sunyc, zhangjq17}@pku.edu.cn

2022 International Conference on Service Science (ICSS) | 978-1-6654-9861-6/22/$31.00 ©2022 IEEE | DOI: 10.1109/ICSS55994.2022.00012

***Abstract*—Software developers are encouraged to reuse mature third-party code repositories to accelerate their developing. The first step is to find some code repositories which satisfy developers' demand for specific functionality. Repository hosting platforms, such as GitHub, NPM, etc. are good search databases. However, these platforms provide little retrieval function based on repositories' functional semantics, which is not conducive for developers to get desired code repository by claing what functionality they require.**

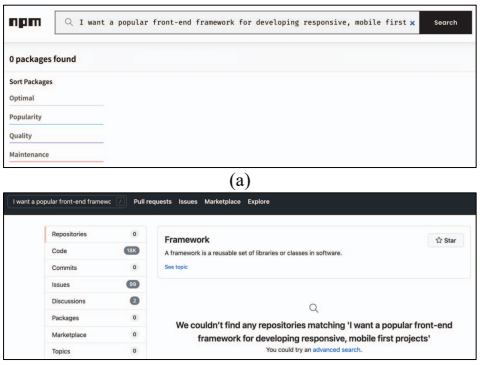
**To solve this problem, we propose and implement an open-source repository retrieval service. First, to extract structured functional semantics of open-source repositories, we design a NER (Named Entity Recognition) model to extract features of six different dimensions related to software development. Next, we construct a HIN (Heterogeneous Information Network) to represent nonstructured semantic information of repositories. Finally, by combining two modules preceding, we implement an open-source repository retrieval service, in which a software developer only needs to tell the service what functionality he (or she) wants and will get a ranked list of repositories as return. We further conduct experiments and the results demonstrate the effectiveness of the service.**

***Keywords—deep learning, named entity recognition, heterogeneous information network, knowledge graph, open-source repository***

## I. INTRODUCTION

With the trend of agile development, instead of developing a software project from scratch, software developers are encouraged to reuse mature third-party repositories to add new functional features to their projects and accelerate their project development. During software reuse, developers always need to know which repositories they can reuse. A clear identification of potential repositories suitable for reuse likely leads to a successful software reuse.

Therefore, software developers face the requirements for finding the repositories which satisfy their demand for specific functionality. However, we identified the problem of little functional semantic-based search service provided by general repository search systems. One of the most widely used repository search system is search engines, such as Baidu, Bing and so on. Software developers input queries into the search engine and get a list of ranked items. But it's not convenient for them to recognize the exact needed items related to software development from the long list. Besides, search engines rarely consider and utilize the functional semantic of code repositories in the retrieval process. The second choice is repository hosting platforms, such as GitHub for open-source projects, NPM for JavaScript projects, and so on. Fortunately, these platforms usually provide topic-based search functions, which reflect parts of the functional semantics of repositories. However, the repository search is still quite limited. GitHub has only 134 featured topics which are too small to cover all functional features of repositories. For NPM, there are many user-defined topic keywords which weaken the precision of search. More importantly, we find that both GitHub and NPM search function rely on exact string matching between queries and target repositories and fail easily. Here is an example, we extract parts of the description of 'bootstrap', a famous front-end CSS framework, and add "I want" at the beginning in NPM as query and request GitHub and NPM. Both websites return nothing, as shown in Fig. 1.




(a)

(b)

Fig. 1. A case of failed search of NPM (a) and GitHub (b)

Instead of modeling semantic information of repository level and providing semantic retrieval for repository, previous studies mainly focus on API level. They usually leverage the information of annotations from code blocks or API document, use deep learning method like [1][2] or design function patterns, like [3] to model or extract the functional semantics for APIs. There are other studies attempting to import external dataset (e.g., Wikidata), like [4] to add a higher level of semantics to API. However, we argue that as API is contained in code repositories, how could developers get an API without knowing what repository they should use. Compared with API recommendation, repository retrieval should be a more basic requirement. Meanwhile, previous API recommendation methods are usually effective when limited in one or several libraries, not suitable for retrieval from large number of repositories.

This paper proposes an open-source repository retrieval service for software developers. The service mainly consists of three modules: a) *Structured Functional Semantics Extraction*, b) *Nonstructured Functional Semantics Extraction*, c) *Repository Retrieval*. In the *Structured Semantics Extraction* module, this service extracts structured functional semantics of open-source repositories from six dimensions, namely programming language, operating

* Corresponding author

978-1-6654-9861-6/22/$31.00 ©2022 IEEE
DOI 10.1109/ICSS55994.2022.00012



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

platform, associated repository, data format, concept related to software development and others involved. The paper proposes a deep learning-based NER model to finish the task. Considering there are other indicative words in repository's description texts which clearly distinguish the repository from others, the service constructs HIN to represent these nonstructured functional words, which is finished in *Nonstructured Functional Semantics Extraction* module. In the last module, the paper designs an open-source repository retrieval approach and implements the corresponding service. Given a developer's query, first, the service extracts named entities and indicative words using preceding models. Then the service outputs a list of repositories as retrieval results by comprehensive consideration of structured functional information and unstructured functional information.

The main contributions of this paper are as follows:

- To the best of our knowledge, this paper is the first to propose functional semantic level mining and modeling for open-source repositories.
- As far as we know, this paper is the first to propose a deep-learning based named entity recognition model for open-source repository to extract functional semantics. The experiments show the model outperforms some baselines.
- This paper puts forward and implements an open-source repository retrieval service based on repositories' functional semantics. Eexperiments are carried out and the results show the service outperforms some baselines.

The rest of the paper is organized as follows. Section 2 introduces a motivating example and inspiration of the service. In Section 3, we introduce related work. Section 4 presents the design and implementation of the service. Section 5 discusses the experiments and results. Section 6 concludes the paper.

## II. Background

### A. Motivating Example

This session presents a motivating example to depict the inspiration of the whole approach and explain the problem this paper aims to solve.

When a developer searches online for reusable third-party repositories to complete specific development tasks, he (or she) probably directly writes down his (or her) demand for specific functionality as a query, like this:

"I want a fast js library to parse pdf into xml."

From this query, three named entities: "js", "pdf", "xml" can be easily identified. They limit the functional semantics of the target repository from different aspects. Named entity "js" requires that the target repository must be written in JavaScript or can be applicable in JavaScript projects, while named entity "pdf" and "xml" requires that target repository must be able to handle files with pdf or xml data formats. These named entities put strict constraints on the possible target repositories.

Except the named entities, there are still words indicating the target repository's unique functional features. For example, the adjective "fast" implies that the target library must execute efficiently, distinguishing them from those, which could accomplish the same functionality, but are too heavy or inefficient. Similar qualifiers include "uniform", "responsive".

Inspired by these observations, this paper regards the semantics implied by the named entities as structured semantics and develops a NER model to extract these semantics from six dimensions including programming languages and data formats. Besides, this paper considers this nonstructured function semantics is mostly expressed by adjectives and nouns. Therefore, this paper collects indicative words with above part-of-speeches from repositories' descriptive texts as a supplementary representation of repository functions. Based on these, we design the whole implementation of our service.

## III. Related Work

This paper focuses on open-source repository retrieval by matching developers' queries and repositories based on functional semantics. This section mainly covers existing literature in four research fields, including open-source repository analysis and data mining, open-source repositories retrieval, development assistant tools.

### A. Open-source Repository Data Mining

At present, GitHub has become the most widely used code repository hosting platform for developers. It has also become an important data source for data mining.

Most of GitHub data mining research focuses on repositories' source files including source codes, README files and so on. Mcmillan et al. proposed a novel approach for automatically detecting similar applications for a given Java application [5]. Gu et al. designed a graph kernel-based approach to selecting API usage examples based on open-source code snippets from GitHub [6]. Zhou et al. proposed a trending repository recommendation service by taking GitHub topics as developers' technical preference [7]. Prana et al. labeled sections in GitHub repository README files and trained a multi-classifier [8].

There are also some studies focusing on analyzing users' behaviors in GitHub. Sinha et al. and Guzman et al. attempted to extract sentiment information from developers' commit log on GitHub [9][10]. Rastogi et al. analyzed the relationship between developers' geographic location and the accept of their pull requests [11]. Liao et al. investigated the features of GitHub issues for better development organization [12]. More related studies on other behaviors like star, watch, fork can be found in [13][14][15].

### B. Open-source Repository Recommendation

Most code hosting platforms such as NPM and GitHub lack standard quality evaluation and strict classification. Therefore, most researches focus on mining original data in these platforms and building a better repository classification system or recommendation system. Sipio et al. utilized the README files and source codes of GitHub repositories and trained a Multinomial Naïve Bayesian (MNB) network to reassign topics to these repositories [16]. For repository or library recommendation, Xu et al. proposed a scalable and personalized approach to recommend projects by leveraging both developers' behaviors and project features [17]. Liu et al. proposed a learning-to-rank modal, neural network for listwise ranking (NNLRank), to recommend projects that developers are likely to contribute [18]. Chunyang Chen et al. presented an approach to recommend analogical libraries by combining word embedding technique and domain-specific relational and categorical knowledge mined from Stack Overflow [19].Yu et al. combined collaborative filtering



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

component to recommend libraries when a new app is given [20]. Jian Chen et al. proposed a third-party library recommendation approach by integrating topic modeling and knowledge graph techniques [21]. Li et al. proposed a graph neural network-based approach for recommending potentially useful third-party libraries for app development [22]. It extracts accurate third-party libraries information through an app-library graph which consists of mobile apps, third-party libraries, and their interactions.

### C. Development Assistant Tools

In addition to recommend the whole library or repository, there are other studies focusing on recommend code snippets [6], code edits and API to assist software development. Zimmerman et al. presented a general framework for recommending reliable and accurate code edits by code analysis based on graph matching and tree edit algorithm [23]. Xie et al. proposed an API recommendation approach based on explicit matching of functional verb phrase in functionality descriptions and user queries, called PreMA [3]. Anas et al. proposed an approach based on frequent usage patterns mining for reengineering object-oriented APIs into component-based one, in order to help developers reuse and understand large APIs [24]. Liu et al. proposed an approach for generating on-demand, hybrid summaries for APIs corresponding with users' queries in specific programming task [4]. Nguyen et al. extracted API usage pattern from java projects and use context-aware collaborative filtering method to recommend API invocation sequence for developers. Gu et al. adopted deep-learning method to model code snippets and their description texts into high-dimensional vector space [1]. Based on the embedding similarity, they provide a code snippet recommendation based in user's queries. And further they extend the method to API level [2].

## IV. Overview of the Repository Retrieval Service

This section presents the repository retrieval service in detail. The service can be divided into two separate modules: online and offline module. Offline module is used to prepare data for online module, while online module is responsible for reacting to a developers' input query and generating a list of repositories as return. In the offline part, the paper proposes an NER model to extract structured functional semantics from six dimension and build a knowledge graph to organize the recognized named entities. Also, this paper develops a HIN to model nonstructured functional semantics about open-source repositories as a supplementary representation. In the online module, we mainly designs a retrieval approach based on the functional semantics and the approach is dedicated to model semantic relationship between developers' query and open-source repositories.

According to Stack Overflow Developer Survey 2021 , JavaScript is the most commonly used programming language. By default, this paper uses open-source JavaScript repositories as main data. These repositories mainly come from NPM (Node Package Manager) websites. Since NPM is part of GitHub, repositories' information in GitHub is also used in this service. In the future, this service could be easily extended to other programming languages.

Fig. 2 illustrates the architecture of the open-source repository retrieval service. The service mainly consists of these parts: *Dataset Preparation* module, Structured *Functional Semantics Extraction* module, *Nonstructured Functional Semantics Extraction* module and *Repository Retrieval* module. This paper will introduce them separately in the following.

### A. Dataset Preparation

As mentioned before, we use JavaScript repositories on GitHub and NPM websites as our data source. They are public and involve nothing about users' privacy. In detail, these data are mainly used for analysis, namely, GitHub REDAME files, GitHub users, GitHub descriptions of repositories, GitHub repository names, NPM description of repositories, NPM keywords assigned to repositories, similar with GitHub topics and NPM domains which are tagged by NPM website to show the development fields of repositories.

Most of the data are well-formed and need no data cleaning, except for GitHub README files. Considering there are many contents irrelevant with the research goals in GitHub README files, we take the following steps to clean README files:

1) Markdown files are converted into plain texts by removing markdown tags and embedded HTML elements in the files. For example, consecutive "#" letters, which stand for title in markdown, are removed.

2) All contents irrelevant with repositories' functionality are removed. There are many paragraphs in README file describing how to use and contribute to the repository. We only keep the first two paragraphs or those with a title containing words like "features", "why choose" and so on. These paragraphs are reckoned to be highly related to the repository's functionality.

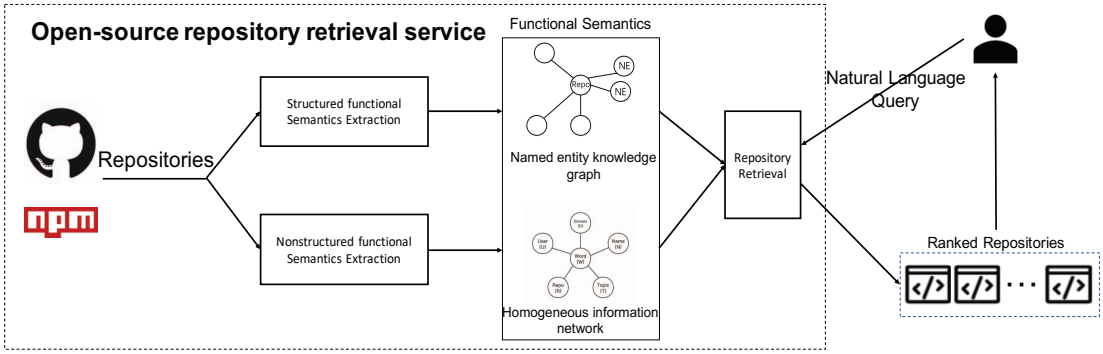



Fig. 2. The architecture of the open-source repository retrieval service



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

3) The remaining contents are separated into sentences.

This paper screens out all immature repositories, i.e., those repositories with less than 10 GitHub stars. And the remaining repositories add up to 20351.

Considering a deep learning-based NER model will be trained, which needs supervised data, we carry out dataset labeling work. Based on the observation and experience in front-end development, we decide to label named entities in GitHub README sentences and GitHub description into following categories:

1) Programming language, which indicates what programming language this repository is written in or applicable in. Some examples are "TypeScript", "js", "CSS" and so on. Note that some modeling language and markup language depicting configurations, for example, yaml, will be included in this category, too.
2) Operating platform, which indicates which platform the repository can be run on. Typical operating platforms include "iOS", "Windows" and so on. Considering that node.js is JavaScript runtime environment built on Chrome's V8 JavaScript engine. In this paper, "node.js" and similar named entities will be classified into this category.
3) Associated repository, mainly consisting of required repositories for running the repository.
4) Data format, mainly consists of file extensions. For example, "xml", "svg" and so on.
5) Concept related with front-end development. The content contained in this category is kind of broad and relatively weakly defined, but mainly can be divided into four subclasses. Subclass 1 mainly includes the concepts in software development process, like "unit test", "CI" (for Continuous Integration). Subclass 2 mainly consists of common standards and techniques which should be followed and used in specific front-end development domain, respectively. Typical examples include "MQTT" (for Message Queuing Telemetry Transport) which is a lightweight, publish-subscribe network protocol, "SSR" (for Server-side Rendering) which is a common technique used in web development. Subclass 3 is mainly composed of concepts describing application types, like "PWA" (for Progressive Web Apps), "UI" (for User Interface). Subclass 4 mainly consists of objects and APIs which frequently appear in some programming languages, like "DOM" (for Document Object Model), "async" which is a keyword in JavaScript.
6) Others, other named entities helping distinguish the repository.

We ask two master students and one undergraduate student to label the data. Every sentence in the dataset will be separately labeled by one master student and the undergraduate student. When encountering phrases which are difficult to label, they are required to consult the search engines to determine whether it is a named entity and which category it should belong to. If their labeling results disagree with each other, the other master student will be involved and discusses until an agreement is reached. In the labeling process, the full names of some abbreviated named entities are also recorded, i.e., JavaScript is the full name of js.

### *B. Structured Functional Semantics Extraction*

In this session, we will introduce how to extract structured functional semantics and how to represent them. We will also introduce how to get named entities and repositories' embedding representation, which will be a requirement of following approach. The whole process can be divided into two parts: 1) NER Model, 2) Structured Semantics Embedding.

***a) NER Model***. Since in our labeled dataset, there is no overlapping between named entities and one named entity only belong to one category, we implement the NER model as a sequence labeling model. Previous studies have shown fine-tuned BERT (Bidirectional Encoder Representation from Transformers) models perform well in many NLP (natural language processing) tasks. Therefore, the NER model is based on BERT [26]. The architecture of the model is shown in Fig. 3.

Before input a sentence to the model, spacy is used[1] to tokenize the sentence into tokens, represented as $w_1, w_2, w_3 \dots w_n$, while $n$ denotes the length of the tokens. Then the tokens are passed to a BERT encoding layer, in which a pretrained BERT model, bert-large-cased[2] is used to get embedding features of each token, represented as $f_1, f_2, f_3 \dots f_n$. To better catch the semantics by aggregating information from different range of neighbor tokens, we import convolution with different dilated sizes as additional feature extraction layers on the top of BERT encoding layer. We call it a multi-scale dilated convolutional layer [27]. Based on experiences, we design three dilated convolution layers with dilated size of 1, 3, 5, respectively paralleled in the multi-scale dilated convolutional layer. Each of the convolution result on the token with index $i$ is represented as $c_i^j$, where $j$ represents the index of different convolution layers. After the multi-scale dilated convolutional layer, we concatenate the embedding vectors output by BERT encoder layer with three additional features got from the convolution layers. By this way, we get the complete semantic features $e_1, e_2, e_3 \dots e_n$ for each token. The complete features are then passed to a BiLSTM layer to get long-distance pattern of tokens and a Conditional Random Field (CRF) layer to add constraints on the legality of tag sequences. Both layers are both popular practices for sequence labeling [28].

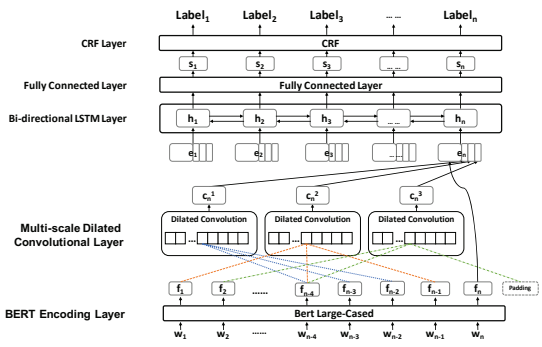



Fig.3. The architecture of the NER model

[1] https://spacy.io/

[2] https://huggingface.co/bert-base-cased



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

***b) Structured Semantics Embedding***. In the dataset preparation section, sentences are tokenized into words, but lemmatization is not carried out. As a result, there may be two phrases with high literal similarity referring to the same named entity. For example, "UIs" is the plural form of "UI", but they almost share same functional semantics and should be resolved as one named entity. To solve this problem, this paper develops a heuristic entity resolution method. Let $NEs = \{NE_1, NE_2, \dots, NE_n\}$ denotes the whole named entities output by preceding NER model, while $n$ represents the number of named entities. For a single named entity $NE_i$, $C_i$ represents the frequency of occurrence of $NE_i$ in the output. We think two named entities $NE_i$ and $NE_j$ should be resolved as one when similarity of their lemmatization is greater than threshold $t$ :

$$\frac{2 * CSL\left(lemma(NE_i), lemma(NE_j)\right)}{len\left(lemma(NE_i)\right) + len\left(lemma(NE_j)\right)} \geq t \qquad (1)$$

$lemma(NE_i)$ represents the lemmatization of $NE_i$, while $CSL(sent1, sent2)$ returns the length of all common sequences in from-left-to-right string matching between $sent1$ and $sent2$. Once two named entities are deemed as same, both named entities will be resolved as the one with bigger $C$ value.

After finishing entity resolution, for modeling the semantic relationship between named entities and open-source repositories, this paper builds a knowledge graph of which the scheme is shown in Fig. 4 The main entities of the graph include recognized named entities with six kinds (namely NE1, NE2, NE3, …NE6), open-source repositories, GitHub users, GitHub topics and NPM keywords. The relations connecting these nodes are illustrated in Fig. 4. Note that for bettering embedding training, we also add "dependency" and "dev-dependency" relations between repositories from NPM websites to the knowledge graph. In the data labeling session, we find many named entities, i.e., JavaScript are often abbreviated as their abbreviations, JS. Considering that qualified names and their abbreviations almost share the same semantic information, we add bidirectional "SameToNE" relations between qualified named entities and their short names. Both relations are not shown in Fig. 4. Before constructing the knowledge graph, we screen out GitHub topics and NPM keywords with low frequency.

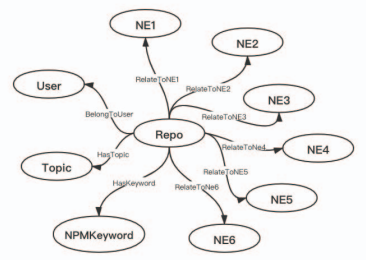



Fig.4. The scheme of the open-source repository knowledge graph

To acquire embedding representation of nodes and relations in the knowledge graph, this paper adapts TransE [29] a knowledge graph embedding algorithm. For a knowledge graph $S$ composed of triples $(h, l, t)$, where $l$ denotes the relation connect head entity $h$ to tail entity $t$, TransE encode the graph by simply supposing:

$$h + l = t \qquad (2)$$

TransE encodes the entities and relations in the knowledge graph into $d$-dimentional vectors by minimizing the loss:

$$L = \sum_{(h,l,t)\in S} \sum_{(h',l,t')\in S'_{(h,l,t)}} [\gamma + d(h + l, t) - d(h' + l, t')] \qquad (3)$$

Where $\gamma$ is the margin hyperparameter, $S'_{(h,l,t)}$ denotes the negative sampling regard triple $(h, l, t)$.

This paper uses OpenKE [30] to implement the TransE model. The embedding size of entities and relations is set to 200.

### *C. Nonstructured Functional Semantics Extraction*

Other than the structured functional semantics reflected by named entities of six different dimensions, we notice that there are other indicative words imposing strong semantic restriction for the search result. HIN[31][32] is a graph which requires that either the number of node types or the number of relation types is larger than 1, and performs well in modeling homogeneous information. This section will introduce HIN to model these unstructured semantics and represent them into embedding space.

***a) HIN construction***. Inspired by HiGitClass [33], we construct HIN in a similar way. We design a HIN of word-center scheme, which is shown in Fig. 5. In the HIN, there are five kinds of nodes. We extract vocabulary of these nodes by the following rules:

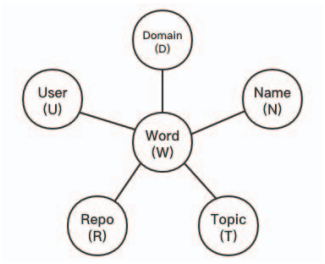



Fig.5. The scheme of our HIN

- User (U): The vocabulary mainly consists of GitHub users of repositories.
- Repo (R): The vocabulary is composed of concrete repositories
- Name (N): This paper segments repository names by "-", "_" and whitespace. The vocabulary is built by summing up all segmented words.
- Topic (T): This paper filters out all GitHub topics and NPM keywords with few occurrences. The Topic vocabulary is composed of remaining GitHub topics and NPM keywords.
- Domain (D): The vocabulary consist of 12 NPM domains indicating the repository is suitable for specific development fields, like front-end.
- Word (W): As these indicative words often takes the part-of-speech of adjective and noun, we extract all these words in repositories' GitHub description, NPM description and README files, filter out words with low frequency and build the initial set. The word vocabulary is constructed by combining the initial set, domain vocabulary and topic vocabulary.



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

After building the vocabularies for the six kinds of nodes, we add edges to the HIN network by this principle: If a word occurs in a repository description texts, and a name happens to occur in the same repository, we add an undirected edge between the word and the name. We follow similar principles to add edges for remaining 4 types of edge.

***b) HIN Embedding.*** Once HIN is constructed, word representations can be learnt by training a HIN embedding. We choose ESim [34] as our HIN embedding algorithm. It's a random walk-based algorithm, but it restricts the walk under the guidance of user-defined meta-paths[31]. A meta-path is an abstraction of paths proposed to describe multi-hop relationships, which can be represented as the node types along the relation path. This paper focuses on acquiring the embedding of words and modeling semantic relationship between words and repositories, so this paper select five meta-paths to guide the ESim training: "W-D-W", "W-N-W", "W-R-W", "W-T-W" and "W-U-W".

The ESim algorithm trains node embedding by maximizing the following likelihood:

$$P(u|v,M) = \frac{\exp\big(f(u,v,M)\big)}{\sum_{v' \in V} \exp\big(f(u,v',M)\big)} \quad (4)$$

Where

$$f(u,v,M) = \mu_M + p_M^T e_u + q_M^T e_v + e_u^T e_v \quad (5)$$

Here, $V$ denotes the node set of the HIN, $v$ and $u$ are single node belonging to $V$. $M$ stands for a meta-path. $\mu_M$ is the global bias of meta-path $M$, $p_M$ and $q_M$ are local bias of meta-path $M$. $e_u$ and $e_v$ are $d$-dimensional embedding vectors for node $u$ and $v$, respectively. (4) represents the conditional probability of node $v$ connected to node $u$ by some path instances following the meta-path $M$.

However, due to the high computational complexity of Equation (1) caused by summing up all nodes' information, in actual computing, this paper estimate the computing through negative sampling:

$$P(u|v,M) = \frac{\exp\big(f(u,v,M)\big)}{\sum_{v' \in V^-} \exp\big(f(u,v',M)\big) + \exp\big(f(u,v,M)\big)} \quad (6)$$

Where $V^-$ is a node set serving as the negative sampling results.

The open-source ESim project[3] is leveraged to implement the HIN embedding. The size of the embedding vectors is set to 100.dddd

### *D. Repository Retrieval*

This section mainly introduces how to search repository based on developers' input. The problem is formulated as following: Given a developer's query $q$, the service returns a ranked list of repositories $Rs$. First, using modules mentioned before, we extract six dimensional named entities for $q$, denoted by $NE_q$, and indicative words, denoted by $KW_q$. Based on this information, we calculate structured and nonstructured functional semantics matching score between a repository $r$ and developer's query $q$:

$$MS_r = \sum_{ne \in NE_q} lk(r,ne) \times \cos_sim(ke_{ne} - ke_{rel}, ke_r) \quad (7)$$

$$MN_r = \sum_{kw \in KW_q} lh(r,kw) \times \cos_sim(he_r, he_{kw}) \quad (8)$$

Here, $lk(r,ne)$ indicate whether repository $r$ and named entity $ne$ is connected in the open-source repository knowledge graph. If they are connected, $lk(r,ne)$ is set to 1 else 0. $lh(r,kw)$, indicating whether repository $r$ and word $kw$ is connected in the HIN, following the same computation pattern.

However, connection information is not enough to model semantic matching between $ne$ and $r$. For example, a general repository contains many named entities in its description texts, while a specific one has less named entities. They may share one common named entity $ne$. However, the semantic emphasis on the $ne$ posed by the two repositories cannot be totally the same. To model this degree of semantic matching, the $lk(r,ne)$ is multiplied with a weight which is computed by cosine similarity between $ke_{ne} - ke_{rel}$ and $ke_r$. Here $ke_{ne}$, $ke_r$, $ke_{rel}$ represent knowledge graph embedding of $ne$, $r$ and the relation connecting $r$ to $ne$, respectively. Following similar pattern as TransE computation, $ke_{ne} - ke_{rel}$ indicates the supposing repository which named entity $ne$ is connected by relation $rel$, while $ke_r$ is the real repository $ne$ is connected to. For $MN_r$, the semantic matching weight is computed by cosine similarity between $he_r$ and $he_{kw}$, representing HIN embedding of repository $r$ and word $kw$.

Once $MS_r$ and $MN_r$ are gotten, the whole matching score between repository r and developers' query q can be calculated as weighted average of the two values:

$$S = \alpha \times MS_r + (1-\alpha) \times MN_r \quad (9)$$

Where $\alpha$ is the weight adjusting the proportion of $MS_r$ and $MN_r$ in the final $S$. The $\alpha$ is set to 0.35. We output a ranked list of repositories with highest $S$ as the retrieval results. To better serve software developers, this paper implements the service as a web application.

## V. EVALUATIONS

The goal of this service is representing structured and nonstructured functional semantics and providing software developers with open-source repository retrieval, given queries claiming what specific functionality they demand for the target repositories. As NER model is a core module of the service, we will introduce the evaluation of its performance first. As the service aims at repository retrieval based on repositories' functional semantics, in this section, the paper will evaluate the effectiveness of our service in repository retrieval.

We propose the following research questions to guide our experiments:

- **RQ1**: Is our proposed NER model well performed enough to fit in the task?
- **RQ2**: How is the performance compared with potential baselines in repository retrieval?

### *A. Named Entity Recognition Experiment*

To answer the **RQ1,** this paper carries out many experiments by configuring different parameters and setting different deep learning models to test if the proposed model is

[3] https://github.com/shangjingbo1226/ESim



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

the best. Since previous studies have shown outperformance of fine-tuned BERT model in NLP tasks, all the models tested bases on BERT pretrained model as well.

***a) Dataset Collection.*** We use NPM dataset collected before. Among it, there are 8012 labeled sentences. This study randomly selects 90% of the labeled sentences as training dataset, while the rest as the validation dataset. All the model mentioned below will be trained solely on training dataset and validated on the validation dataset.

***b) Metric Selection.*** Precision, recall and F1 metrics are used to evaluate the performance of NER models. Before introducing the metrics, we give the following definition:

- True Positive (TP): The NER model labels some phrases as named entities with specific category c, and they happen to be contained in ground-truth of the same category.
- False Positive (FP): The NER model labels some phrases as named entities with specific category c, but they are not contained in the ground-truth or contained in the ground-truth but with a different category.
- False Negative (FN): The ground-truth contains some named entities which are not labeled out by the NER model.

The precision is defined to measure the rate of the correct named entities in the total named entities predicted by NER model:

$$Precision = \frac{TP}{TP + FP} \tag{10}$$

The recall is defined to measure the rate of the correct named entities in the total named entities in the ground-truth.

$$Recall = \frac{TP}{TP + FN} \tag{11}$$

F1 can be computed by:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{12}$$

***c) Experimental Setting.*** First, we want to test which strategy of data preprocessing gets the best results. We use NER model described before but changes the strategy of data preprocessing. The first is doing nothing, which is tagged as "Model-Normal" in the following. The second is switching the text data into lower case, as we believe it could reduce the size of word vocabulary. In the following, NER model adopting this strategy of data preprocessing is tagged as "Model-Lower". The third strategy is stemming every word in the text data, as we consider it could disambiguate named entities sharing the same root forms, such as "UI" and its plural form "UIs" before input into the model. This strategy is tagged as "Model-Lemma". Pretrained BERT models is adjusted for different preprocessing strategies, i.e., bert-large-cased[4] for Strategy Model-Normal, bert-large-uncased[5] for Strategy Model-Lower and Strategy Model-Lemma.

Second, we want to validate whether every part of the NER model is effective and whether the whole model achieve the best performance. An ablation study is conducted. The details of the selected models are depicted below:

- BERT-simple: We remove all middle layers and CRF layer. As a result, the BERT encoding layer is directly connected to the fully connected layer.
- BERT-BiLSTM: Compared with BERT-simple model, we add a BiLSTM layer between BERT encoding layer and output layer.
- BERT-DialatedCNN-BiLSTM: Compared with the complete model depicted before, only remove CRF layer is removed.
- BERT-whole: the complete model depicted before.

All models sharing the same training parameters. The experiment uniformly sets batch size to 32, learning rate to $1e^{-5}$ (for CRF layer and fully connected layer, $1e^{-3}$), training epochs to 20.

***d) Results and Analysis.*** For the first experiment, the results are shown in Table I.

TABLE I. PERFORMANCE OF DIFFERENT DATA PREPROCESSING STRATEGIES IN NER

| Preprocess Strategy | Precision | Recall | F1 |
|---|---|---|---|
| Model-Normal | **0.835** | **0.862** | **0.848** |
| model-Lower | 0.826 | 0.850 | 0.838 |
| Model-Lemma | 0.803 | 0.819 | 0.810 |

We find that strategy of doing nothing achieves the best performance. We explain that the case of letters already carries a lot of semantic information. A letter with upper case probably implies a beginning of the sentence or named entity. Strategy 2 erases these semantics, which causes its lower performance compared to Strategy 1. As for Strategy 3, after a word is inputted to the BERT encoding layer, it will further be segmented into word pieces, which kind of disambiguates named entities sharing the same root forms. So, the Strategy 3 sounds useless.

For the second experiment, the result is shown in Table II.

TABLE II. PERFORMANCE OF DIFFERENT MODELS IN NER

| Model | Precision | Recall | F1 |
|---|---|---|---|
| BERT-simple | 0.372 | 0.848 | 0.517 |
| BERT-BiLSTM | 0.704 | 0.768 | 0.734 |
| BERT-DialatedCNN-BiLSTM | 0.602 | 0.800 | 0.687 |
| BERT-whole | **0.835** | **0.862** | **0.848** |

By comparing performance of BERT-simple and BERT-BiLSTM, BERT-DialatedCNN-BiLSTM and BERT-whole, we find that sequence layer like BiLSTM and CRF layer helps to improve precision significantly. We explain that they are better at characterizing dependencies between sequential information. For example, CRF layer could learn transformation between labels and help correcting wrong labeling, such as label "B" followed by label "S" which is illegal in BIOS labeling principle, thus, improves precision. We also find that introduction of feature extraction layers, like DialatedCNN and BERT encoding layers, improve recall. In

[4] https://huggingface.co/bert-base-cased

[5] https://huggingface.co/bert-base-uncased



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

summary, our proposed model achieves considerable performance.

### B. Repository Retrieval Experiment

To answer the **RQ2**, we conduct experiments by evaluating accuracy of repositories output by the service given developers' queries. In the following, we will introduce *Dataset Collection*, *Metrics Selection*, *Baseline Selection*, *Experimental Setting* and *Results and Analysis* sequentially.

***a) Baseline Selection.*** To the best of our knowledge, there is no any similar research work focusing on providing repository retrieval based on software developers' queries, which contain their demand for specific functionality. Also, because of poor performance of GitHub and NPM when a developer's query does not strictly match the target repositories' descriptions, they are not suitable for evaluation. Therefore, we choose general search engine, i.e., Bing as the first kind of baseline. To demonstrate service's better performance, we develop two baseline approaches based on Doc2Vec and Tf-Idf, respectively.

For the Doc2Vec [35] based baseline, we use the same dataset, which is composed of description texts of repositories and gensim [36] is used to train the Doc2Vec. Based on the cosine similarity of developers' queries and all repositories, we select top-n repositories with highest cosine similarity between them and developers' queries.

For the Tf-Idf based baseline, same dataset is used. After getting Tf-Idf vector of all repositories and users' query, we follow the similar pattern as Doc2Vec baseline to give top-n repositories as retrieval results.

***b) Dataset Collection.*** Considering the distinctiveness of the approach, we need to simulate developers' input query. To generate developers' queries, we select description texts of 15 repositories from their official websites or corresponding Wikipedia pages, other than GitHub and NPM as data source. To simulate the scene where a software developer enters the query, a master student is asked to reorganize the description text in the data source. In this process, some information may be pruned, replaced by similar one, or added. One another master student is asked to evaluate the similarity between reorganized texts and original texts, to guarantee that reorganized texts still convey the semantic information as original texts. For example, the description text of bootstrap in Wikipedia is "Bootstrap is a free and open-source CSS framework directed at responsive, mobile-first front-end web development." maybe transferred into "I want a responsive and mobile-first CSS framework in web development". We regard the reorganized texts as queries, and corresponding repositories as exact answers.

***c) Metrics Selection.*** We use commonly used recommender system metrics to evaluate performance of our approach and the baselines.

The first metric is Success rate. For a test dataset, Success Rate is defined to measure the rate at which the answer exists in the top-20 repositories.

The second metric is MRR (Mean Reciprocal Rank) which is a widely used recommender system. In the context of this paper, it is calculated by this formula:

$$MRR = \frac{1}{|R|}\sum_{i=1}^{|R|}\frac{1}{rank_i} \tag{13}$$

Where $R$ denotes the testing dataset, $rank_i$ represents the position of the answer repository in the searched results for test sample $i$.

***d) Experimental Setting.*** We want to test our approach's performance comparing with baselines mentioned before. Note that for search engine, the experiment has two different configurations. By default, we consider search engines find the answer only when official websites link to GitHub link or NPM link of the target repository appears in the result list. Of course, all advertisement link output is excluded in the result list. In the extended configuration, as long as the hyperlink ouputted by search engines points to an article which contains introduction of the target repository, it is deemed a successful retrieval.

***e) Results and Analysis.*** The results are shown in Table III.

TABLE III. THE EVALUATION RESULTS IN REPOSITORY RETRIEVAL APPROACH EXPERIMENT

| Model | Success Rate | MRR |
|---|---|---|
| Our Approach | **86.7%** | **0.541** |
| Bing-default | 46.7% | 0.178 |
| Doc2Vec | 20.0% | 0.038 |
| Tf-Idf | 60.0% | 0.296 |
| Bing-extended | 80.0% | 0.550 |

No matter in default setting or in extended setting, our service achieves the best performance. Note that we consider nothing with repositories' popularity as search engine do. In the extended setting, any hyperlinks outputted by search engines which contains an introduction to the target repository is deemed as a successful retrieval. This condition is quite loose and still our service outperforms Bing. In summary, our service achieves considerable performance. This also confirms the effectiveness of service.

## VI. CONCLUSION AND FUTURE WORK

This paper identifies the problem of the shortage of functional semantic based repository retrieval provided by current repository search system, including GitHub, NPM and search engines. To alleviate this problem, the paper proposes an open-source repository retrieval service. To extracting structured functional semantics, this paper puts forward to deep-learning-based NER model to extract named entities reflecting six-dimensional semantics related with software development. To represent the remaining nonstructured functional semantics, this paper develops a HIN and trains a HIN embedding. Combining these two modules, this paper implements the service. And Experiments demonstrate the service overperforms some baselines, showing our service's effectiveness. In the future, we plan to refine the nonstructured semantics extraction method and extend the service to other programming languages.


## ACKNOWLEDGMENT

This effort is sponsored by the Beijing Outstanding Young Scientist Program under grant number BJJWZYJH01201910001004.


## REFERENCES



Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.

[1] Gu X, Zhang H, Kim S. Deep code search[C]//2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE). IEEE, 2018: 933-944.

[2] Gu X, Zhang H, Zhang D, et al. Deep API learning[C]//Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. 2016: 631-642.

[3] Xie W, Peng X, Liu M, et al. API method recommendation via explicit matching of functionality verb phrases[C]//Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2020: 1015-1026.

[4] Liu M, Peng X, Marcus A, et al. Generating query-specific class API summaries[C]//Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2019: 120-130.

[5] C, Grechanik M, Poshyvanyk D. Detecting similar software applications[J]. Proceedings - International Conference on Software Engineering, 2012.

[6] X Gu, Zhang H , Kim S . CodeKernel: A Graph Kernel Based Approach to the Selection of API Usage Examples[C]// 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE). ACM, 2019 : 590-601.

[7] Zhou Y, Wu J, Sun Y. GHTRec: A Personalized Service to Recommend GitHub Trending Repositories for Developers[C]//2021 IEEE International Conference on Web Services (ICWS). IEEE, 2021: 314-323.

[8] Prana G, Treude C, Thung F, et al. Categorizing the Content of GitHub README Files[J]. Empirical Software Engineering, 2018.

[9] Vinayak Sinha, Alina Lazar, and Bonita Sharif. 2016. Analyzing developer sentiment in commit logs. In Proceedings of MSR 2016 (13th International Conference on Mining Software Repositories). ACM, 520--523.

[10] Emitza Guzman, David Azócar, and Yang Li. 2014. Sentiment analysis of commit comments in GitHub: an empirical study. In Proceedings of MSR 2014 (11th Working Conference on Mining Software Repositories). ACM, 352--355.

[11] A. Rastogi, N. Nagappan, G. Gousios, and A. van der Hoek. Relationship between geographical location and evaluation of developer contributions in github. In Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM '18, pages 22:1–22:8, New York, NY, USA, 2018. ACM.

[12] Liao Z, Dayu H, Chen Z, et al. Exploring the Characteristics of Issue-Related Behaviors in GitHub Using Visualization Techniques[J]. IEEE Access, 2018:24003-24015.

[13] Ma W, Lin C, Zhang X, et al. How Do Developers Fix Cross-Project Correlated Bugs? A Case Study on the GitHub Scientific Python Ecosystem[C]// 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE). ACM, 2017.

[14] Kalliamvakou, E., et al. The promises and perils of mining GitHub. In MSR, ACM (2014).

[15] J. T. Tsay, L. Dabbish, and J. Herbsleb. Social media and success in open source projects. In Proc. Computer Supported Cooperative Work Companion, pages 223–226, 2012.

[16] Sipio C D, Rubei R, Ruscio D D, et al. A Multinomial Nave Bayesian (MNB) Network to Automatically Recommend Topics for GitHub Repositories[C]// EASE '20: Evaluation and Assessment in Software Engineering. 2020.

[17] Wenyuan Xu, Xiaobing Sun, Xin Xia, Xiang Chen. Scalable relevant project recommendation on GitHub. In: Proceedings of the 9th Asia-Pacific Symposium on Internetware, Shanghai, 2017

[18] Liu C, Yang D, Zhang X, et al. Recommending GitHub Projects for Developer Onboarding[J]. IEEE Access, 2018, 6:52082-52094.

[19] Chen C . Mining Analogical Libraries in Q&A Discussions -- Incorporating Relational and Categorical Knowledge into Word Embedding[C]// IEEE International Conference on Software Analysis. IEEE, 2016.

[20] Huan Yu, Xin Xia, Xiaoqiong Zhao, Weiwei Qiu et al. Combining Collaborative Filtering and Topic Modeling for More Accurate Android Mobile App Library Recommendation[C]// the 9th Asia-Pacific Symposium. 2017.

[21] Chen J , Li B , Wang J , et al. Knowledge Graph Enhanced Third-Party Library Recommendation for Mobile Application Development[J]. IEEE Access, 2020, PP(99):1-1.

[22] Bo Li, Qiang He, Feifei Chen, Xin Xia, Li Li, John Grundy, and Yun Yang. 2021. Embedding app-library graph for neural third party library recommendation. In Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) 2021: 466–477.

[23] Zimmerman K , Rupakheti C R . An Automated Framework for Recommending Program Elements to Novices (N)[C]// IEEE/ACM International Conference on Automated Software Engineering. ACM, 2016: 283-288.

[24] Shatnawi A , Seriai A , Sahraoui H , et al. Mining Software Components from Object-Oriented APIs[C]// International Conference on Software Reuse. Springer, Cham, 2015:330-347.

[25]

[26] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding[J]. arXiv preprint arXiv:1810.04805, 2018.

[27] Yu F, Koltun V. Multi-scale context aggregation by dilated convolutions[J]. arXiv preprint arXiv:1511.07122, 2015.

[28] Huang Z, Xu W, Yu K. Bidirectional LSTM-CRF models for sequence tagging[J]. arXiv preprint arXiv:1508.01991, 2015.

[29] Bordes A, Usunier N, Garcia-Duran A, et al. Translating embeddings for modeling multi-relational data[J]. Advances in neural information processing systems, 2013, 26.

[30] Han X, Cao S, Lv X, et al. Openke: An open toolkit for knowledge embedding[C]//Proceedings of the 2018 conference on empirical methods in natural language processing: system demonstrations. 2018: 139-144.

[31] Sun Y, Han J. Mining heterogeneous information networks: principles and methodologies[J]. Synthesis Lectures on Data Mining and Knowledge Discovery, 2012, 3(2): 1-159.

[32] Sun Y, Han J, Yan X, et al. Pathsim: Meta path-based top-k similarity search in heterogeneous information networks[J]. Proceedings of the VLDB Endowment, 2011, 4(11): 992-1003.

[33] Zhang Y, Xu F F, Li S, et al. Higitclass: Keyword-driven hierarchical classification of github repositories[C]//2019 IEEE International Conference on Data Mining (ICDM). IEEE, 2019: 876-885.

[34] Shang J, Qu M, Liu J, et al. Meta-path guided embedding for similarity search in large-scale heterogeneous information networks[J]. arXiv preprint arXiv:1610.09769, 2016.

[35] Le Q, Mikolov T. Distributed representations of sentences and documents[C]//International conference on machine learning. PMLR, 2014: 1188-1196.

[36] Rehurek R, Sojka P. Software framework for topic modelling with large corpora[C]//In Proceedings of the LREC 2010 workshop on new challenges for NLP frameworks. 2010.




Authorized licensed use limited to: University of Michigan Library. Downloaded on October 11,2026 at 21:50:26 UTC from IEEE Xplore. Restrictions apply.